



sample deliverable

MVP Blueprint

The written plan behind Relaydesk, a client approval portal: users, features with a clear “done”, data model, security, architecture, decisions and running costs.

This is what the Basic package delivers before any code is written. Relaydesk is a fictional sample product.

Sample project. Relaydesk is a fictional product built as a public portfolio demo. Every company, person and figure in it is invented. Payments run in Stripe test mode.

1. The product in one paragraph

Small creative and marketing agencies send work to clients (logos, ad sets, landing pages, videos) and then chase approvals over email, WhatsApp and shared drives. Feedback gets lost, nobody knows which version is final, and invoices wait on a "looks good" that never arrives in writing. Relaydesk gives each client a private space where the agency posts versioned deliverables, the client approves or requests changes from any device, and every decision is recorded with a timestamp.

2. Users and roles

Role	Who	Can
Owner	Agency founder	Everything in the workspace, including billing and deleting it
Member	Agency staff	Manage clients, deliverables and comments. No billing.
Client	A person at the client company	See only their own client space. Comment, approve, request changes.
Platform admin	Relaydesk staff	Read-only view of all workspaces, the webhook log and the audit log. Can suspend a workspace.

The platform admin flag lives in Supabase `app_metadata` (only the server can set it), never in `user_metadata` (users can edit that).

3. Features and "done means"

#	Feature	Done means
1	Workspaces, team and client invites	Sign up creates a workspace with you as Owner. Owners invite Members and Clients by link (expires after 7 days, single use, only the token's hash is stored). A Client sees only their own space, and a test proves it.
2	Deliverables with versions	A Member uploads a file to a client space. Uploading again creates v2, v3, and older versions stay viewable. Files sit in a private bucket and open through short-lived signed URLs.
3	Approvals and comments	A Member requests approval with a due date. The Client approves or requests changes, from a phone as easily as a laptop. Comments thread under each version. Every decision lands in the activity log.

#	Feature	Done means
4	Billing	Free, Pro and Studio plans through Stripe Checkout and the Customer Portal. Plans change only from Stripe's own data: a verified webhook (signature checked against the raw body, deduplicated on the event id), or a pull from Stripe's API when someone returns from Checkout. Limits are enforced on the server and the UI explains them.
5	Admin panel	Platform admins see workspaces, plans, test-mode MRR, the webhook event log (processed and duplicates skipped) and the audit log, and can suspend a workspace.
6	AI revision checklist	One click turns a feedback thread into a checklist. Each item quotes the comment it came from. The output is labelled as AI-generated, and the feature is rate-limited and Pro-plan only.
7	Pins on the work	A client taps an image to pin a numbered note to that spot. Pins stay on their version, link to their comment, and the studio resolves them. Demo copies arrive with earlier pins.
8	Compare versions	A slider wipes between two image versions; it works with the keyboard and screen readers.
9	Sign-off sheet	A printable record of an approval (who, which version, when, every round before it) with the SHA-256 fingerprint of the exact approved file.
10	Search, nudges, notifications	Ctrl/⌘K jumps to any deliverable, client or page. Work waiting on a client gets a ready-made reminder. A bell shows what others did since you last looked.

The \$600 MVP package in the gig maps to features 1 to 3; the \$1,500 Full launch package maps to all six. Features 7 to 10 show what a polished V1 adds on top.

4. Plans

	Free	Pro	Studio
Price (test mode)	\$0	\$29 / month	\$79 / month
Client spaces	2	15	Unlimited
Team members	2	10	Unlimited
Storage	250 MB	20 GB	100 GB
AI revision checklist		50 / month	300 / month

5. Data model

```
auth.users (Supabase)
└─ profiles          display name
workspaces          a studio (tenant): plan, demo flag, suspended_at
├─ memberships      user + role (owner | member | client)
├─ invitations       one-time link: token hash, role, expiry
├─ clients           a client space ("Northwind Coffee")
│   └─ deliverables  title, status, due date
│       ├── deliverable_versions  v1, v2...: file path, size, type
│       ├── reviews              approve / request changes, per version
│       ├── comments             per version; optional pin (x, y), resolved
│       └─ revision_checklists    AI to-do lists
│           └─ checklist_items    task, client's quote, source comment
├─ activity          what happened, and who may see it
├─ notification_reads  each person's "seen up to"
└─ subscriptions     Stripe customer, status, period end
stripe_events        one row per Stripe event id (dedupe, admin log)
audit_log            platform admin actions (server only)
demo_sandboxes       demo copies, for the 24-hour cleanup
```

Deliverable status is a small state machine: `draft` → `in_review` → (`changes_requested` or `approved`).

Uploading a new version moves `approved` or `changes-requested` work back to `draft`, so the studio asks for approval again on the new file.

6. Security plan

1. **Row Level Security on every table**, with policies written per access path, not one copy-pasted

`auth.uid() = user_id` rule.

2. **Explicit, least-privilege grants.** New Supabase tables are no longer exposed to the Data API automatically (changelog, 2026-04-28). Each table grants only the verbs its policies need, to

`authenticated` only. `anon` gets nothing.

3. **Membership checks in one place.** Small `SECURITY DEFINER` helper functions

(`private.workspace_role()`, `private.can_access_client()`) live in a schema the API doesn't expose, take no user-supplied ids they don't verify, and are indexed. Policies call them as (`select ...`) so Postgres evaluates them once per query.

4. **Storage paths carry the tenant:** `{workspace_id}/{client_id}/{deliverable_id}/v{n}-{file}`.

Storage policies check those folders against the same helpers.

5. **Server-side auth uses `getClaims()`**, which verifies the JWT. `getSession()` is never trusted on the server.

6. **The service role key never reaches the browser.** It is used only on the server: the Stripe webhook and billing sync, creating and cleaning up demo copies, and the admin panel.

7. **Tests prove isolation.** pgTAP tests log in as a Client of workspace A and assert they can't read workspace B, another client's space in A, or unapproved internal data.

7. Architecture

```
Browser —> Vercel (Next.js 16, App Router, Node runtime)
    |
    |— proxy.ts           refreshes the Supabase session cookie
    |— Server Components  read data as the signed-in user (RLS applies)
    |— Server Actions      validate with zod, then write as the user
    |— /api/stripe/webhook verifies signature, dedupes, writes with service role
    |— /api/cron/cleanup-demo daily: deletes demo copies over 24 h old
    |
    —> Supabase (Postgres + RLS, Auth, private Storage)
    —> Stripe (test mode: Checkout, Customer Portal, webhooks)
    —> Google Gemini, free tier (feature 6; Vercel AI Gateway also supported)
```

8. Decisions (short ADRs)

- 1. **Supabase over a separate auth + database + storage stack.** One service, one row-level security model covering both table rows and files. Trade-off: business logic leans on Postgres policies, so they are tested like code.
- 2. **Server Components + Server Actions, no separate API layer.** Fewer moving parts for an MVP. The only route handlers are for things that must be HTTP endpoints (webhook, cron).
- 3. **Stripe is the source of truth for plans.** The webhook pushes every change; coming back from Checkout or the billing portal pulls the same state from Stripe's API, so the plan is right at once even if a webhook is late or never arrives. Both go through one idempotent database function. The browser never grants anything.
- 4. **Invites by link first, email second.** The link works with no email provider configured. Email delivery switches on when an SMTP or Resend key is present.
- 5. **Demo mode: a private sandbox per visitor, not a shared demo account.** A shared demo login lets one visitor approve everything, delete files, post junk the next visitor sees, or change the shared password through the auth API. Instead, "Try the demo" creates a fresh copy of the demo agency for that visitor: four demo users, seeded clients, files and history. A "View as client" switch signs them into the client side of *their own* copy. Switching uses a server-generated one-time sign-in link, so the demo users have no known passwords. Copies are deleted after 24 hours, creation is rate-limited per IP, and uploads in demo workspaces are capped. Real sign-ups get their own empty workspace and never see demo data.

9. Monthly running cost

Users	Vercel	Supabase	Stripe	AI	Total
Demo / 0	\$0 (Hobby)	\$0 (Free)	\$0	about \$0	\$0
100 agencies	\$20 (Pro, commercial use)	\$25 (Pro)	2.9% + 30¢ per payment	about \$5	about \$50 + Stripe fees
1,000 agencies	\$20 + usage	\$25 + usage (about \$10 to \$40)	same	about \$40	about \$120 to \$150 + Stripe fees

Vercel's Hobby plan is for non-commercial use, so a real launch moves to Pro.

10. Out of scope (said up front)

Native mobile apps, invoicing and payments between agency and client, real-time co-editing, white-label custom domains, SSO.